

MATERI PEMROGRAMAN C++

Class & Object

OOP: Konsep, Atribut, Method, Konstruktor, Destruktor, Enkapsulasi, Inheritance, dan Polymorphism

Bahasa	Paradigma	Tingkat	Topik Utama
C++	OOP	Pemula - Menengah	Class & Object

1. Pengantar Pemrograman Berorientasi Objek (OOP)

Object-Oriented Programming (OOP) adalah paradigma pemrograman yang mengorganisir kode berdasarkan **objek** — entitas yang memiliki data (atribut) dan perilaku (method). C++ adalah salah satu bahasa yang mendukung OOP secara penuh.

Empat Pilar OOP:

Pilar	Konsep	Manfaat
Encapsulation	Menyembunyikan detail implementasi	Keamanan & modularitas data
Inheritance	Kelas anak mewarisi kelas induk	Reusabilitas kode
Polymorphism	Satu antarmuka, banyak bentuk	Fleksibilitas & extensibility
Abstraction	Menyajikan hanya hal yang penting	Menyederhanakan kompleksitas

2. Class dan Object

Sebuah **Class** adalah blueprint (cetak biru) yang mendefinisikan struktur dan perilaku suatu objek. **Object** adalah instance (realisasi) dari sebuah class.

■ *Analogi: Class adalah 'resep kue', Object adalah 'kue yang sudah jadi'. Satu resep bisa menghasilkan banyak kue dengan bahan berbeda.*

Sintaks Dasar Class:

```
class NamaClass {
public:
    // Atribut (data member)
    tipe_data nama_atribut;

    // Method (member function)
    tipe_kembali nama_method(parameter);
};

// Membuat object
NamaClass namaObject;
```

Contoh Class Pertama:

```
#include <iostream>
#include <string>
using namespace std;

class Mahasiswa {
```

```

public:
    string nama;
    int    nim;
    double ipk;

    void tampilkan() {
        cout << "Nama : " << nama << endl;
        cout << "NIM  : " << nim  << endl;
        cout << "IPK  : " << ipk  << endl;
    }
};

int main() {
    Mahasiswa mhs1;           // membuat object
    mhs1.nama = "Budi";
    mhs1.nim  = 12345;
    mhs1.ipk  = 3.75;
    mhs1.tampilkan();
    return 0;
}

```

3. Access Specifier

Access specifier menentukan siapa yang boleh mengakses anggota class. Ada tiga jenis di C++:

Specifier	Akses dari Class Sendiri	Akses dari Class Turunan	Akses dari Luar Class
public	Ya	Ya	Ya
protected	Ya	Ya	Tidak
private	Ya	Tidak	Tidak

```

class Contoh {
public:
    int publik;           // bisa diakses dari mana saja

protected:
    int terlindung;      // hanya class ini & turunannya

private:
    int rahasia;         // hanya class ini saja
};

```

■ *Best practice: jadikan atribut sebagai private/protected, dan akses melalui method public (getter/setter). Inilah inti dari Enkapsulasi!*

4. Konstruktor dan Destruktor

Konstruktor adalah method khusus yang dipanggil otomatis saat object dibuat. **Destruktor** dipanggil otomatis saat object dihancurkan.

a) Default Constructor:

```
class Mobil {
public:
    string merk;
    int    tahun;

    // Default constructor (tanpa parameter)
    Mobil() {
        merk  = "Unknown";
        tahun = 2000;
        cout << "Mobil dibuat!" << endl;
    }

    // Destructor
    ~Mobil() {
        cout << "Mobil dihapus!" << endl;
    }
};
```

b) Parameterized Constructor:

```
class Mobil {
public:
    string merk;
    int    tahun;

    // Constructor dengan parameter
    Mobil(string m, int t) {
        merk  = m;
        tahun = t;
    }
};

int main() {
    Mobil m1("Toyota", 2022);
    Mobil m2("Honda", 2023);
    return 0;
}
```

c) Constructor Initialization List (Direkomendasikan):

```
class Mobil {
public:
    string merk;
    int    tahun;

    // Initialization list – lebih efisien
    Mobil(string m, int t) : merk(m), tahun(t) {}
};
```

d) Constructor Overloading:

```
class Lingkaran {
public:
    double radius;

    Lingkaran()          : radius(1.0) {} // default
    Lingkaran(double r)  : radius(r)   {} // dengan radius
};

int main() {
    Lingkaran l1;          // radius = 1.0
    Lingkaran l2(5.5);     // radius = 5.5
    return 0;
}
```

5. Getter & Setter — Enkapsulasi

Enkapsulasi menyembunyikan data (private) dan menyediakan method publik untuk membaca (getter) dan mengubah (setter) nilai atribut. Ini menjaga integritas data.

```
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private:
    string pemilik;
    double saldo;

public:
    BankAccount(string p, double s) : pemilik(p), saldo(s) {}

    // Getter
    string getPemilik() const { return pemilik; }
    double getSaldo()   const { return saldo;   }
```

```
// Setter dengan validasi
void setor(double jumlah) {
    if (jumlah > 0) saldo += jumlah;
    else cout << "Jumlah tidak valid!" << endl;
}

void tarik(double jumlah) {
    if (jumlah > 0 && jumlah <= saldo)
        saldo -= jumlah;
    else
        cout << "Saldo tidak cukup!" << endl;
}

};

int main() {
    BankAccount akun("Ani", 1000000);
    akun.setor(500000);
    akun.tarik(200000);
    cout << "Saldo: " << akun.getSaldo() << endl; // 1300000
    return 0;
}
```

6. Inheritance (Pewarisan)

Inheritance memungkinkan sebuah class (child/derived) mewarisi atribut dan method dari class lain (parent/base). Ini mendorong reusabilitas kode.

```
#include <iostream>
#include <string>
using namespace std;

// --- Base class (Kelas Induk) ---
class Hewan {
protected:
    string nama;
    int    umur;

public:
    Hewan(string n, int u) : nama(n), umur(u) {}

    void makan() {
        cout << nama << " sedang makan." << endl;
    }

    void info() {
        cout << "Nama: " << nama << ", Umur: " << umur << endl;
    }
};

// --- Derived class (Kelas Turunan) ---
class Anjing : public Hewan {
private:
    string ras;

public:
    Anjing(string n, int u, string r)
        : Hewan(n, u), ras(r) {}

    void menggonggong() {
        cout << nama << ": Guk guk!" << endl;
    }
};

int main() {
    Anjing d("Buddy", 3, "Labrador");
    d.info();           // diwarisi dari Hewan
    d.makan();          // diwarisi dari Hewan
    d.menggonggong();   // method milik Anjing sendiri
}
```



```

    return 0;
}

```

Jenis-jenis Inheritance di C++:

Jenis	Sintaks	public member	protected member
Public	class B : public A	public	protected
Protected	class B : protected A	protected	protected
Private	class B : private A	private	private

7. Polymorphism

Polymorphism berarti 'banyak bentuk'. Sebuah method dapat berperilaku berbeda tergantung objek yang memanggilnya. Ada dua jenis:

a) Compile-time Polymorphism — Function Overloading:

```

class Kalkulator {
public:
    int    tambah(int a, int b)          { return a + b; }
    double tambah(double a, double b)   { return a + b; }
    int    tambah(int a, int b, int c)   { return a + b + c; }
};

int main() {
    Kalkulator k;
    cout << k.tambah(2, 3)              << endl;  // 5
    cout << k.tambah(2.5, 1.5)          << endl;  // 4.0
    cout << k.tambah(1, 2, 3)          << endl;  // 6
    return 0;
}

```

b) Runtime Polymorphism — Virtual Function:

```

class Bentuk {
public:
    virtual void gambar() { // keyword 'virtual'
        cout << "Menggambar bentuk" << endl;
    }
    virtual ~Bentuk() {}    // selalu buat virtual destructor
};

class Lingkaran : public Bentuk {

```

```

public:
    void gambar() override { // keyword 'override' (C++11)
        cout << "Menggambar lingkaran" << endl;
    }
};

class Persegi : public Bentuk {
public:
    void gambar() override {
        cout << "Menggambar persegi" << endl;
    }
};

int main() {
    Bentuk* b1 = new Lingkaran();
    Bentuk* b2 = new Persegi();

    b1->gambar(); // Output: Menggambar lingkaran
    b2->gambar(); // Output: Menggambar persegi

    delete b1;
    delete b2;
    return 0;
}

```

■ Gunakan 'override' (C++11) untuk memastikan method benar-benar meng-override virtual function dari class induk. Compiler akan memberi error jika tidak cocok.

8. Abstract Class dan Pure Virtual Function

Abstract class adalah class yang tidak bisa dibuat objeknya secara langsung. Ia mendefinisikan antarmuka (interface) yang harus diimplementasikan oleh class turunan melalui **pure virtual function** (ditandai dengan = 0).

```
class Bentuk {                // Abstract class
public:
    // Pure virtual function
    virtual double luas()      const = 0;
    virtual double keliling() const = 0;

    void cetak() {             // method biasa boleh ada
        cout << "Luas: "      << luas()      << endl;
        cout << "Keliling: " << keliling() << endl;
    }
    virtual ~Bentuk() {}
};

class Lingkaran : public Bentuk {
private:
    double r;
public:
    Lingkaran(double r) : r(r) {}
    double luas()      const override { return 3.14159 * r * r; }
    double keliling() const override { return 2 * 3.14159 * r; }
};

class Persegi : public Bentuk {
private:
    double sisi;
public:
    Persegi(double s) : sisi(s) {}
    double luas()      const override { return sisi * sisi; }
    double keliling() const override { return 4 * sisi; }
};

int main() {
    // Bentuk b; // ERROR: tidak bisa buat objek abstract class
    Lingkaran l(7);
    Persegi p(5);
    l.cetak();
    p.cetak();
}
```

```
    return 0;
}
```

9. Static Member

Anggota **static** adalah milik class, bukan milik object tertentu. Semua object berbagi satu salinan dari static member.

```
class Counter {
private:
    static int jumlah;    // dihitung bersama semua object

public:
    Counter() { jumlah++; }
    ~Counter() { jumlah--; }

    static int getJumlah() { return jumlah; }
};

// Definisi di luar class (wajib untuk static member)
int Counter::jumlah = 0;

int main() {
    cout << Counter::getJumlah() << endl;    // 0
    Counter a, b, c;
    cout << Counter::getJumlah() << endl;    // 3
    return 0;
}
```

10. Friend Function dan Friend Class

Fungsi atau class yang dideklarasikan sebagai **friend** dapat mengakses anggota private dan protected dari class yang bersangkutan.

```
class Kotak {
private:
    double panjang;

public:
    Kotak(double p) : panjang(p) {}

    // Deklarasi friend function
    friend double hitungVolume(Kotak k, double lebar, double tinggi);
};

// Friend function – bisa akses panjang yang private
double hitungVolume(Kotak k, double lebar, double tinggi) {
```

```
        return k.panjang * lebar * tinggi;
    }

    int main() {
        Kotak k(5.0);
        cout << hitungVolume(k, 3.0, 2.0) << endl; // 30
        return 0;
    }
```

■ *Gunakan friend dengan hati-hati — terlalu banyak friend dapat merusak prinsip enkapsulasi. Pertimbangkan getter/setter terlebih dahulu.*

11. Operator Overloading

C++ memungkinkan kita mendefinisikan ulang cara kerja operator (+, -, *, ==, dll.) untuk tipe data buatan sendiri.

```
class Vektor2D {
public:
    double x, y;

    Vektor2D(double x=0, double y=0) : x(x), y(y) {}

    // Overload operator +
    Vektor2D operator+(const Vektor2D& v) const {
        return Vektor2D(x + v.x, y + v.y);
    }

    // Overload operator ==
    bool operator==(const Vektor2D& v) const {
        return (x == v.x && y == v.y);
    }

    // Overload operator << untuk output
    friend ostream& operator<<(ostream& os, const Vektor2D& v) {
        os << "(" << v.x << ", " << v.y << ")";
        return os;
    }
};

int main() {
    Vektor2D v1(1, 2), v2(3, 4);
    Vektor2D v3 = v1 + v2;
    cout << v3 << endl;           // (4, 6)
    cout << (v1 == v2) << endl;   // 0 (false)
    return 0;
}
```

12. Studi Kasus — Sistem Perpustakaan

Berikut contoh penerapan class dan object dalam kasus nyata — sistem manajemen buku perpustakaan sederhana.

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;
```

```

class Buku {
private:
    string judul, pengarang;
    int    tahun;
    bool    tersedia;

public:
    Buku(string j, string p, int t)
        : judul(j), pengarang(p), tahun(t), tersedia(true) {}

    string getJudul()    const { return judul; }
    bool    isTersedia() const { return tersedia; }

    void pinjam() {
        if (tersedia) {
            tersedia = false;
            cout << judul << " berhasil dipinjam." << endl;
        } else {
            cout << judul << " sedang dipinjam orang lain." << endl;
        }
    }

    void kembalikan() { tersedia = true; }

    void tampilkan() const {
        cout << "[" << (tersedia ? "Tersedia" : "Dipinjam") << "]" ";
        cout << judul << " - " << pengarang
            << " (" << tahun << ")" << endl;
    }
};

class Perpustakaan {
private:
    string        nama;
    vector<Buku>    koleksi;

public:
    Perpustakaan(string n) : nama(n) {}

    void tambahBuku(const Buku& b) { koleksi.push_back(b); }

    void tampilkanSemua() const {
        cout << "=== " << nama << " ===" << endl;
        for (const auto& b : koleksi) b.tampilkan();
    }
};

```

```
int main() {  
    Perpustakaan lib("Perpustakaan Kota");  
    lib.tambahBuku(Buku("C++ Primer", "Lippman", 2012));  
    lib.tambahBuku(Buku("Clean Code", "Martin", 2008));  
    lib.tambahBuku(Buku("Design Patterns", "GoF", 1994));  
    lib.tampilkanSemua();  
    return 0;  
}
```


13. Kesalahan Umum (Common Mistakes)

Kesalahan	Penyebab	Solusi
Lupa semicolon setelah '}'	<code>class A { } // tanpa ;</code>	Selalu akhiri definisi class dengan ;
Akses member private dari luar	<code>obj.nilaiPrivate = 5;</code>	Gunakan setter publik
Tidak ada virtual destructor	<code>class Base { ~Base(){} }</code>	Tambahkan virtual <code>~Base() {}</code>
Memory leak dengan new	<code>Base* p = new Derived(); // tanpa delete</code>	Selalu delete atau gunakan smart pointer
Slicing object	<code>Base b = derivedObj;</code>	Gunakan pointer/referensi: <code>Base* b = &derivedObj</code>
Lupa definisi static member	<code>int Counter::jumlah;</code> tidak ada	Definisikan static member di luar class

14. Latihan Soal

- [Mudah]** Buat class Persegi dengan atribut private sisi. Lengkapi dengan konstruktor, getter/setter, dan method hitung luas() serta keliling().
- [Mudah]** Buat class Mahasiswa dengan atribut nama, nim, dan nilai. Buat method untuk menampilkan grade (A/B/C/D/E) berdasarkan nilai.
- [Sedang]** Buat class hierarchy: class Kendaraan (induk) dengan atribut merk dan kecepatan, lalu class Mobil dan Motor sebagai turunannya dengan atribut tambahan masing-masing.
- [Sedang]** Implementasikan class Stack sederhana menggunakan array dengan method push(), pop(), peek(), dan isEmpty().
- [Sulit]** Buat sistem kasir sederhana menggunakan class Produk, class Keranjang, dan class Transaksi. Implementasikan enkapsulasi, dan tampilkan struk belanja dengan total harga.

Ringkasan

Konsep	Kata Kunci / Sintaks
Definisi class	<code>class NamaClass { ... };</code>
Membuat object	<code>NamaClass obj;</code> atau <code>NamaClass obj(arg);</code>
Access specifier	<code>public / protected / private</code>
Constructor	<code>NamaClass(param) : member(val) {}</code>
Destructor	<code>~NamaClass() {}</code>
Getter const	<code>tipe getNama() const { return nama; }</code>

Inheritance	class Anak : public Induk { ... };
Virtual function	virtual void method() { }
Override (C++11)	void method() override { }
Pure virtual	virtual void method() = 0;
Static member	static tipe nama; — definisi di luar class
Operator overloading	Tipe operator+(const Tipe& t) const { }
Friend function	friend tipe fungsi(param);

■ *OOP adalah fondasi dari software engineering modern. Kuasai class dan object dengan baik sebelum melanjutkan ke template, STL, dan design patterns dalam C++.*